

Intermediate (watch the model choose between two tools) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper LangChain/Python), **Advanced** (the measurement questions a real evaluation would ask). Read Basic first even if you already know this — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

You'll build two separate tools that sound almost the same — like one that gets today's weather, and one that gets a multi-day forecast.

Register both at once, and ask something deliberately vague, like "what's the weather like" — nothing that clearly says "today" or "this week."

Run that same question several times, and write down, each time, which tool got picked. You're not trying to "fix" anything yet — just observe.

Things to use:

- Two `@tool` -decorated functions with similar one-line docstrings.
- `model_with_tools = ChatOpenAI().bind_tools([get_weather, get_forecast])`
- A loop that calls `.invoke("what's the weather like")` 5 times.
- Each time, print `response.tool_calls[0]["name"]` if there is a call, or `"none"` if there isn't.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

`bind_tools([tool_a, tool_b])` hands the model both descriptions at once, and tool selection happens entirely inside the model — your code has no `if / else` deciding which one to call. When both descriptions could plausibly answer the same question, the choice becomes sensitive to small wording differences in the prompt, and can vary run to run even at `temperature=0` for tool selection specifically (temperature affects text generation more predictably than tool-choice, which is worth confirming for yourself here).

The point of running the same ambiguous prompt 5 times isn't to find "the bug" — it's to see, directly, that an overlapping pair of descriptions produces a real, measurable split, not a single wrong answer you can just patch.

Write the two docstrings deliberately similar — e.g. `"Get the current weather for a city."` and `"Get the weather forecast for a city."` — close enough that a human skimming them might also be unsure. Tally the picks with a plain dictionary, like `{"get_weather": 0, "get_forecast": 0, "none": 0}`, and print a real number at the end, not just "it seemed to favor one."

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

A 5-run split like `{"get_weather": 3, "get_forecast": 2}` raises a harder question than "which one is right": **is that split itself a problem, or is it just what a genuinely unclear prompt should produce?** The prompt genuinely doesn't say whether "today" or "this week" was meant — a perfect model might legitimately split its answers too, the same way two different humans reading the same vague request might make different reasonable guesses. The fix isn't always "make the split go away" — sometimes it's "recognize the prompt actually needs a follow-up question," which is a UX decision, not a tool-description bug.

Two things a real evaluation would check that a single ad hoc run never does:

Does the order you register the tools in bias the pick? `bind_tools([get_weather, get_forecast])` and `bind_tools([get_forecast, get_weather])` register the exact same two tools — if the split changes meaningfully just from swapping that order, that's a real, separate bias worth knowing about, independent of the description-wording problem this exercise is about.

Is 5 runs even enough to trust? A 3/2 split and a 30/20 split both "look like" 60/40, but only one of them is a sample size anyone should draw a conclusion from. This exercise uses 5 runs to keep things fast to run by hand — a real evaluation harness would use a much larger N (or a fixed, larger eval set) before treating any split as a real signal instead of noise.

The extra pieces needed to check the order-bias question:

- A second trial, identical except for the order of the list passed to `bind_tools(...)`.
- Comparing the two splits side by side, not just running one and assuming it generalizes.

Sketch what a meaningfully different split (order bias) vs. a noisy-but-similar split (no real bias) would look like, before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate get you to a real, measured split from one trial. Advanced questions whether that one trial is trustworthy at all — checking whether the *order* tools are registered in silently shifts the split (a bug you'd never notice from a single run), and naming the sample-size problem honestly instead of over-trusting 5 runs. This is the difference between "I ran it once and wrote down what happened" and "I checked whether what happened was actually about the descriptions."

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
define get_weather - "get the current weather for a city"
define get_forecast - "get the weather forecast for a city"

register both on a model

counts = {get_weather: 0, get_forecast: 0, none: 0}

repeat 5 times:
    ask "what's the weather like"
    look at which tool (if any) was picked
    add one to that count

print counts
```

Here's almost the whole thing — just try running it:

```
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI

@tool
def get_weather(city: str) -> str:
    """Get the current weather for a city."""
    return f"Sunny in {city}"

@tool
def get_forecast(city: str) -> str:
    """Get the weather forecast for a city."""
    return f"Rainy later this week in {city}"

model_with_tools = ChatOpenAI().bind_tools([get_weather, get_forecast])

counts = {"get_weather": 0, "get_forecast": 0, "none": 0}
for i in range(5):
    response = model_with_tools.invoke("what's the weather like")
    picked = response.tool_calls[0]["name"] if response.tool_calls else "none"
    counts[picked] = counts.get(picked, 0) + 1
```

```
print(counts)
```

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
@tool
def get_weather(city: str) -> str: "Get the current weather for a city."

@tool
def get_forecast(city: str) -> str: "Get the weather forecast for a city."

function get_model_with_tools():
    return ChatOpenAI(...).bind_tools([get_weather, get_forecast])

function run_selection_trial(prompt, runs=5) -> Counter:
    model_with_tools = get_model_with_tools()
    tally = Counter()
    for run_number in range(1, runs + 1):
        response = model_with_tools.invoke(prompt)
        picked = response.tool_calls[0]["name"] if response.tool_calls else "none"
        print(f"Run {run_number}: picked {picked}")
        tally[picked] += 1
    return tally
```

Wire this into a full script with `if __name__ == "__main__":`, then compare against the [Solution](#).

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

```
function get_model_with_tools(tool_order):
    return ChatOpenAI(...).bind_tools(tool_order)

function run_selection_trial(prompt, tool_order, runs=5) -> Counter:
    (same as Intermediate, but built from get_model_with_tools(tool_order))

split_a = run_selection_trial(prompt, [get_weather, get_forecast])
split_b = run_selection_trial(prompt, [get_forecast, get_weather])

print("Registered [weather, forecast] first:", dict(split_a))
print("Registered [forecast, weather] first:", dict(split_b))
```

Here's almost the whole thing — fill in the missing piece yourself:

```
from collections import Counter
from langchain_core.tools import BaseTool, tool
from langchain_openai import ChatOpenAI

@tool
def get_weather(city: str) -> str:
    """Get the current weather for a city."""
    return f"Sunny in {city}"

@tool
def get_forecast(city: str) -> str:
    """Get the weather forecast for a city."""
    return f"Rainy later this week in {city}"

def get_model_with_tools(tool_order: list[BaseTool]) -> ChatOpenAI:
    return ChatOpenAI(model="gpt-4o-mini", temperature=0).bind_tools(tool_order)

def run_selection_trial(prompt: str, tool_order: list[BaseTool], runs: int = 5) -> Counter:
    model_with_tools = get_model_with_tools(tool_order)
    tally: Counter = Counter()
    for run_number in range(1, runs + 1):
        response = model_with_tools.invoke(prompt)
        picked = response.tool_calls[0]["name"] if response.tool_calls else "none"
```

```
tally[picked] += 1  
return tally
```

```
prompt = "what's the weather like"
```

```
# your turn: run the trial twice, once with [get_weather, get_forecast] and once  
# with the order reversed, then print both splits side by side  
...
```

Fill in the order-swap comparison yourself, then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: same underlying trial (register, ask 5 times, tally) at 3 completeness levels — Basic's version prints one dictionary and stops, Intermediate wraps it in a reusable, per-run-logging function, and Advanced runs that same function twice with the registration order swapped, turning "here's a split I got once" into "here's whether that split was actually about the descriptions, or partly about the order I happened to list them in."

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)